

Topic

Recursion Tree method:

⇒ Recursion tree method is a pictorial representation of an iteration method which is in the form of a tree where at each level nodes are expanded.

⇒ it is useful when the divide & conquer algorithm is used.

⇒ A Recursion tree is a tree where each node represents the cost of a certain recursive sub-problem.

⇒ To get the cost of the entire algorithm we sum up the values.

Example:

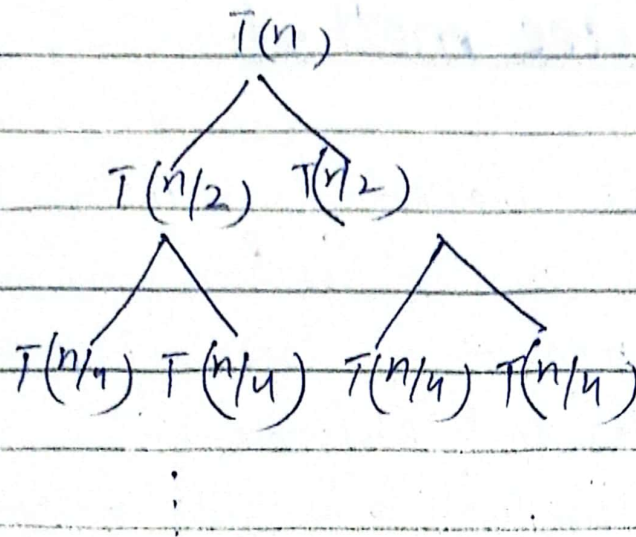
$$T(n) = 2T(n/2) + n$$

Step-1:

○ A problem of size n will get divided into 2 sub-problems of size $n/2$.

○ Each sub-problem of size $n/2$ will get divided into 2 sub-problems of size $n/4$ and so on.

○ At the bottom most layer, the size of sub problems will reduce to 1.



Step-1
 $T(1) \dots \dots T(1)$

Determine the cost of each level

Cost of level 0 = n

Cost of level 1 = $n/2 + n/2 = n$

Cost of level 2 = $n/4 + n/4 + n/4 + n/4 = n$

Step-2

Determine total numbers of levels in the recursion tree

Size of sub-problem at level 0 = $n/2^0$
 at level 1 = $n/2^1$
 at level 2 = $n/2^2$

Size of sub-problem at level $i = n/2^i$
 $\frac{n}{2^i} = 1$
 $2^i = n$

$\log_2 2^i = \log_2 n$
 $i \cdot \log_2 2 = \log_2 n \Rightarrow i = \frac{\log_2 n}{\log_2 2} = \log_2 n$

Total no of levels in the recursion tree = $\log_2 n + 1$

Step-3:

Determine number of nodes in the last level

Level-0 has 2^0 nodes = 1

Level-1 has 2^1 nodes = 2

Level-2 has 2^2 nodes = 4

Step-4:

Last level nodes = $2^{\log_2 n}$ nodes = n

Step-5:

Cost of last level = $n \times T(1)$
 $= \Theta(n)$

Step-6

Add costs of all the levels of the recursion tree

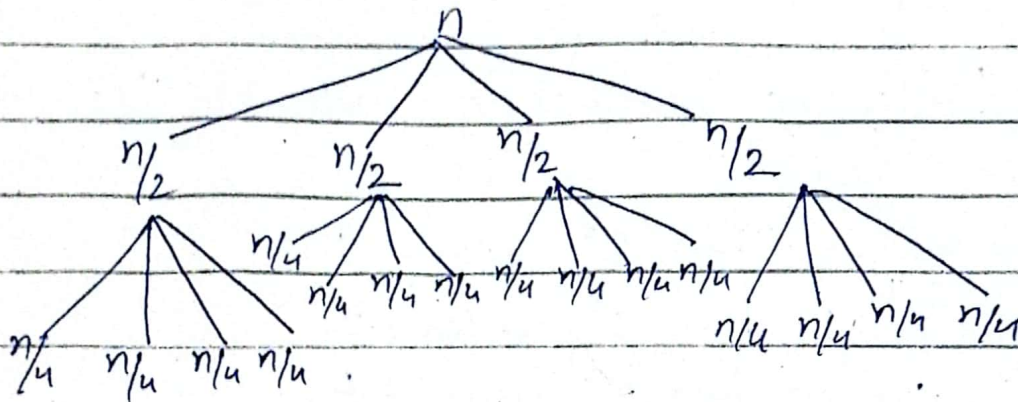
$$T(n) = \{n + n + n + \dots\} + \Theta(n)$$

$$= n \times \log_2 n + \Theta(n)$$

$$= n \log_2 n + \Theta(n)$$

$$= \Theta(n \log_2 n)$$

Solve $4T(n/2) + n$.



$T(1) \dots \dots \dots T(1)$

Step-1:

value of node at $L_0 = n/2^0 = n$

value of node at $L_1 = n/2^1 = n/2$

value of node at $L_2 = n/2^2$

value of node at i^{th} level $= n/2^i$

value of node at last level $= 1$

$$\frac{n}{2^i} = 1$$

$$2^i n = 2^i$$

$$\log n = i \log 2$$

$$i = \frac{\log n}{\log 2} = \log_2 n$$

Step 2:

no of nodes at level 0 $= 4^0 = 1$

no of nodes at level 1 $= 4^1 = 4$

no of nodes at $L_2 = 4^2 = 16$

no of nodes at i^{th} level = 4^i
 no of nodes at last level = $4^{\log_2 n}$
 $= n^{\log_2 4}$
 $= n^2$

Step 3:

Cost at $L=0 = n$

Cost at $L_1 = 2n$

Cost at $L_2 = 4n$

Cost at last level = $n^2 \times T(1) = n^2$

$T(n) = n + 2n + 4n + \dots + n^2$

$= n(1 + 2 + 4 + \dots + n^{\frac{1}{2}})$

$= n(2^0 + 2^1 + 2^2 + \dots + 2^{\log_2 n})$

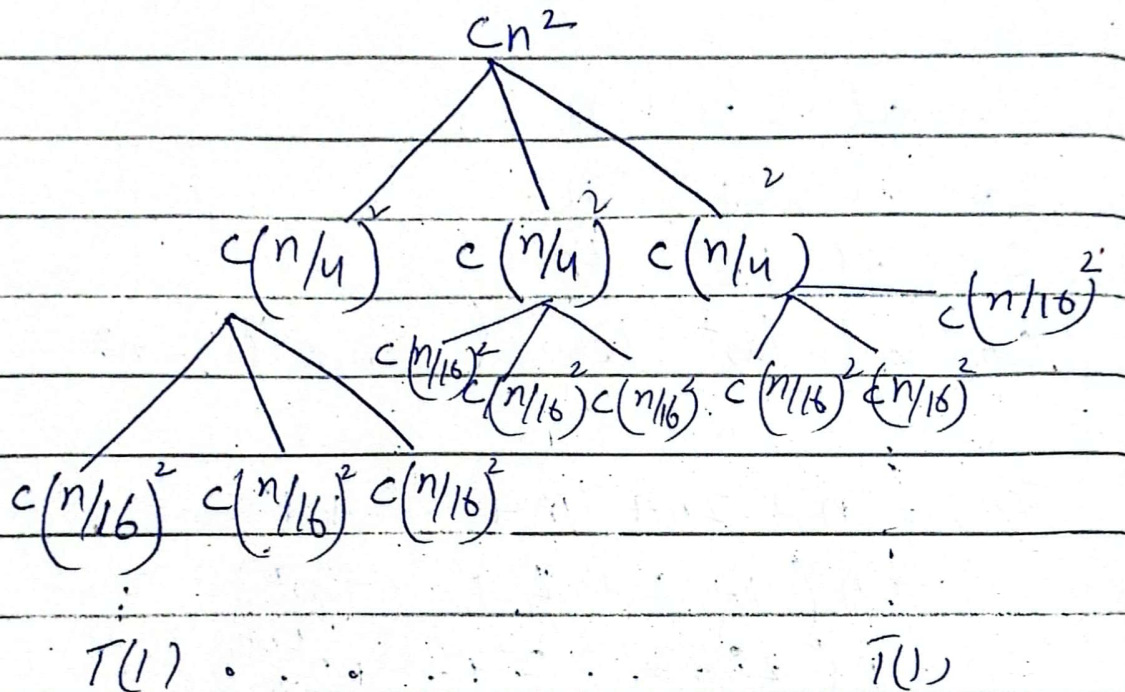
$= \frac{n(2^{\log_2 n} - 1)}{2 - 1}$ G.P

$= n(n - 1)$

$= n^2 - n \in O(n^2)$

Solve using recursion tree method.

$$T(n) = 3T(n/4) + cn^2$$



$\Rightarrow$ value of node at i th level $= \frac{n}{4^i}$
 $\Rightarrow$ value of node at last level $= 1$
 $\frac{n}{4^i} = 1$

$$n = 4^i$$

$$\log n = i \log 4$$

$$i = \frac{\log n}{\log 4}$$

$$i = \log_4 n$$

$$\text{Total no of levels} = \log_4 n + 1$$

no of nodes at i^{th} level = 3^i
 no of nodes at last level = $3^{\log_4 n}$
 $= n^{\log_4 3}$

$$\text{cost at last level} = n^{\log_4 3} \times T(1) \\ = n^{\log_4 3}$$

$$\text{Cost at } L=0 = cn^2$$

$$\text{Cost at } L=1 = (3/16)cn^2$$

$$\text{Cost at } L=2 = cn^2 (9/16^2)$$

$$T(n) = \{cn^2 + \frac{3}{16}cn^2 + \frac{9}{(16)^2}cn^2 + \dots + n^{\log_4 3}\}$$

$$= cn^2 \left\{ 1 + \frac{3}{16} + \left(\frac{3}{16}\right)^2 + \dots \right\} + \dots + n^{\log_4 3}$$

$$= \left\{ 1 + (3/16) + (3/16)^2 + \dots \right\} \text{ G.P infinite}$$

$$= \sum_{i=0}^{\log_4 n - 1} (3/16)^i cn^2 + n^{\log_4 3}$$

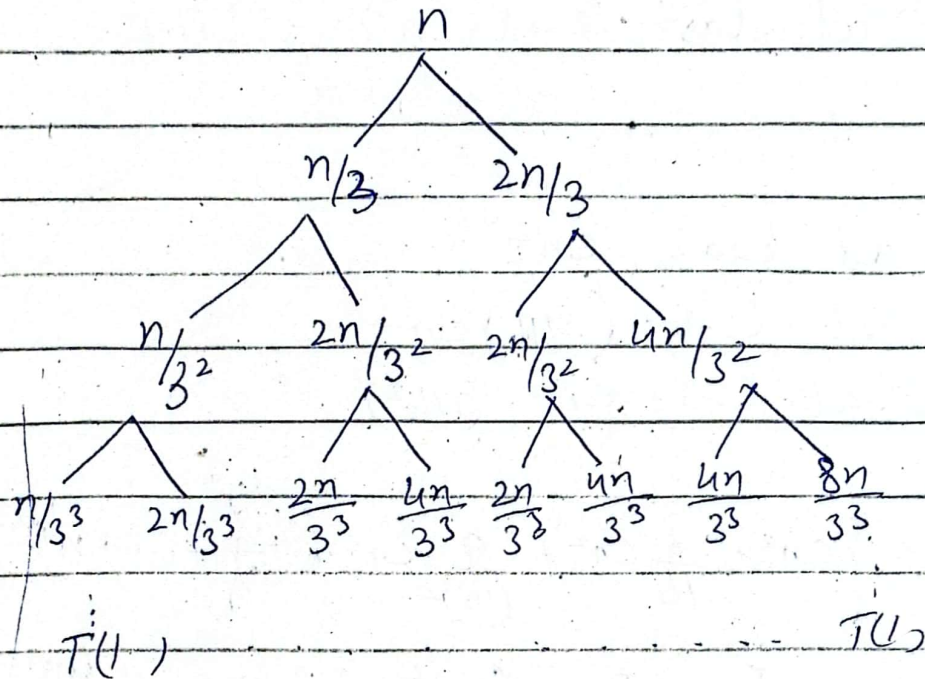
$$= \sum_{i=0}^{\infty} (3/16)^i cn^2 + n^{\log_4 3} \quad \therefore \sum_{i=0}^{\infty} x^i = \frac{1}{1-x}$$

$$= \frac{1}{1-(3/16)} cn^2 + n^{\log_4 3}$$

$$= \frac{1}{13/16} cn^2 + n^{\log_4 3} = \frac{16}{13} cn^2 + n^{\log_4 3} \\ \in O(n^2)$$

Solve using recursion tree method.

$$T(n) = T(n/3) + T(2n/3) + n$$



$\therefore$ Largest value of node at i^{th} level $= (2/3)^i n$

$\therefore$ Value of node at last level = 1

$$n \left(\frac{2}{3} \right)^i = 1$$

$$\left(\frac{2}{3} \right)^i = 1/n$$

$$\left(\frac{3}{2} \right)^i = n$$

$$i \log(3/2) = \log n$$

$$i = \frac{\log n}{\log(3/2)} = \log_{3/2} n$$

∴ Total no of levels = $\log_{3/2} n + 1$

∴ no of nodes at i^{th} level = 2^i

∴ no of node at last level = $2^{\log_{3/2} n}$

Cost at last level = $2^{\log_{3/2} n} \times T(1)$
 $= n^{\log_{3/2} 2}$

$T(n) = \sum_{i=0}^{\log_{3/2} n - 1} n + n^{\log_{3/2} 2}$

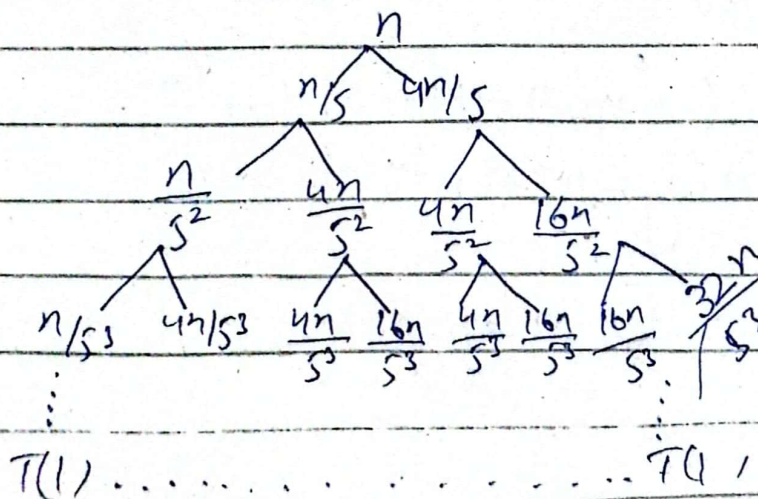
$= n \sum_{i=0}^{\log_{3/2} n - 1} (1) + n^{\log_{3/2} 2}$

$= n (\log_{3/2} n - 1 - 0 + 1) + n^{\log_{3/2} 2}$

$= n \log_{3/2} n + n^{\log_{3/2} 2}$
 $\in O(n \log_{3/2} n)$

Solve using recurrence relation

$T(n) = T(n/5) + T(4n/5) + n$



$$\Rightarrow \text{Largest value of node at } i^{\text{th}} \text{ level} = \left(\frac{4}{5}\right)^i n$$

$$\Rightarrow \text{Value of node at last level} = 1$$

$$\left(\frac{4}{5}\right)^i n = 1$$

$$\left(\frac{4}{5}\right)^i = \frac{1}{n}$$

$$\left(\frac{5}{4}\right)^i = n$$

$$i \log\left(\frac{5}{4}\right) = \log n$$

$$i = \log_{5/4} n$$

$$\Rightarrow \text{Total no of nodes at last level} = 2^{\log_{5/4} n}$$

$$\Rightarrow \text{Cost of node at last level} =$$

$$2^{\log_{5/4} n} \times T(1)$$

$$= n^{\log_{5/4} 2}$$

$$T(n) = \sum_{i=0}^{\log_{5/4} n - 1} n + n^{\log_{5/4} 2}$$

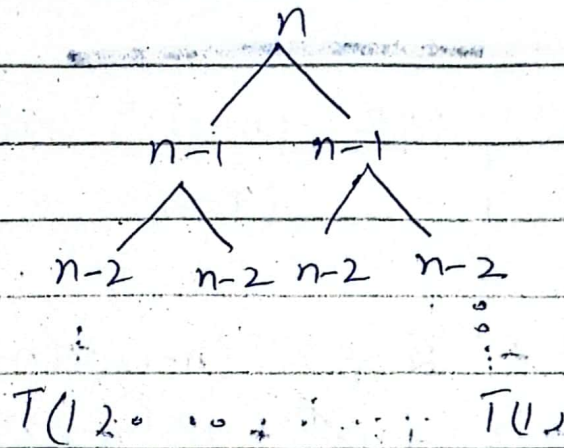
$$= n (\log_{5/4} n - 1 + 1 - 0) + n^{\log_{5/4} 2}$$

$$= n \log_{5/4} n + n^{\log_{5/4} 2}$$

$$\in O(n \log_{5/4} n)$$

Solve using recursion tree

$$T(n) = 2T(n-1) + n$$



$\Rightarrow$ value of node at i^{th} level $= (n-i)$

$\Rightarrow$ value of node at last level $= 1$

$$n-i=1$$

$$i = n-1$$

$\Rightarrow$ no. of nodes at i^{th} level $= 2^{i-1}$

$\Rightarrow$ no. of nodes at last level $= 2^{n-1}$

$$\begin{aligned} \Rightarrow \text{cost at last level} &= 2^{n-1} \times T(1) \\ &= 2^{n-1} \end{aligned}$$

$\Rightarrow$ cost at L-0 $= n$

$\Rightarrow$ cost at L-1 $= 2n-2$

$\Rightarrow$ cost at L-2 $= 4n-8$

$$T(n) = \sum_{i=0}^{(n-1)-1} (2^i n - 2^i \cdot i) + 2^{n-1}$$

$$= \sum_{i=0}^{(n-1)-1} (n-i) 2^i + 2^{n-1}$$

$$\therefore \sum_{i=0}^{n-1} (b+id) a^i = b \sum_{i=0}^{n-1} a^i + d \sum_{i=0}^{n-1} i a^i$$

$$= n \left[\frac{1-2^{n-1}}{1-2} \right] + (1)2 - (n-1)2^{n-1} + (n-2)2^{n-1+1} + 2^{n-1}$$

$$= n(2^{n-1} - 1) + 2 + 2^{n-1}(n-1) + (n-2)2^n + 2^{n-1}$$

$$= n2^{n-1} - n + 2 + n2^{n-1} + 2^{n-1} + n2^n + 2 \cdot 2^n + 2^{n-1}$$

$$= -2 - n + 2 \cdot 2^{n-1} + n2^n + 2 \cdot 2^n$$

$$= -2 - n + 2 \cdot 2^{n-1}(1-2) + n2^n$$

$$= -2 - n + 2 \cdot 2^{n-1}(-1) + n2^n$$

$$= -2 - n - 2 \cdot 2^{n-1} + n2^n$$

$$\in O(2^n)$$

$$= 2 \cdot 2^{n-1} - n - 2 \cdot 2^{n-1} + n2^n$$

$$a^n n(1+1) - n$$

$$\therefore \sum_{i=0}^{n-1} a^i = \frac{1-a^n}{1-a}$$

$$\therefore \sum_{i=0}^{n-1} i a^i = \frac{a - na^n + (n-1)a^{n+1}}{(1-a)^2}$$

$$= n \left[1 - 2^{(n-1)-1} \right]$$

$$= T(n) = \sum_{i=0}^{n-2} 2^i n - \sum_{i=0}^{n-2} 2^i i + 2^{n-1}$$

$$= n \sum_{i=0}^{n-2} 2^i - \sum_{i=0}^{n-2} 2^i i + 2^{n-1}$$

$$= n \left[\frac{1 - 2^{n+1}}{1 - 2} \right] - \left[\frac{2 - (n-1)2^{n-1} + (n-2)2^{n-1}}{(1-2)^2} \right] + 2^{n-1}$$

$$= n (2^{n-1} - 1) - [2 - 2^{n-1}(n-1) + (n-2)2^{n-1}] + 2^{n-1}$$

$$= n2^{n-1} - n - 2 + n2^{n-1} - 2^{n-1} - n2^{n-1} + 2 \cdot 2^{n-1} + 2^{n-1}$$

$$= n2^{n-1} - n + n2^{n-1} - n2^{n-1} + 2 \cdot 2^{n-1} - 2$$

$$\in O(2^n)$$

$$\therefore \sum_{i=0}^{n-1} a^i = 1 - a^n$$

$$\therefore \sum_{i=0}^{n-1} i a^i = \frac{a - n a^n + (n-1) a^{n+1}}{(1-a)^2}$$

2^{n+1}

- Let V A be any matrix then
 trace of a matrix A is denoted
 by $\text{tr}(A)$ defined as the sum
 of entries of main diagonal
 if A is not square
 matrix then its trace cannot be
 find.
 non Singular $\neq |A| \neq 0$

$$E = \begin{bmatrix} 6 & 1 & 3 \\ -1 & 1 & 2 \\ 2 & -1 & 3 \end{bmatrix}, D = \begin{bmatrix} 1 & 5 & 2 \\ -1 & 0 & 1 \\ 3 & 2 & 4 \end{bmatrix}$$

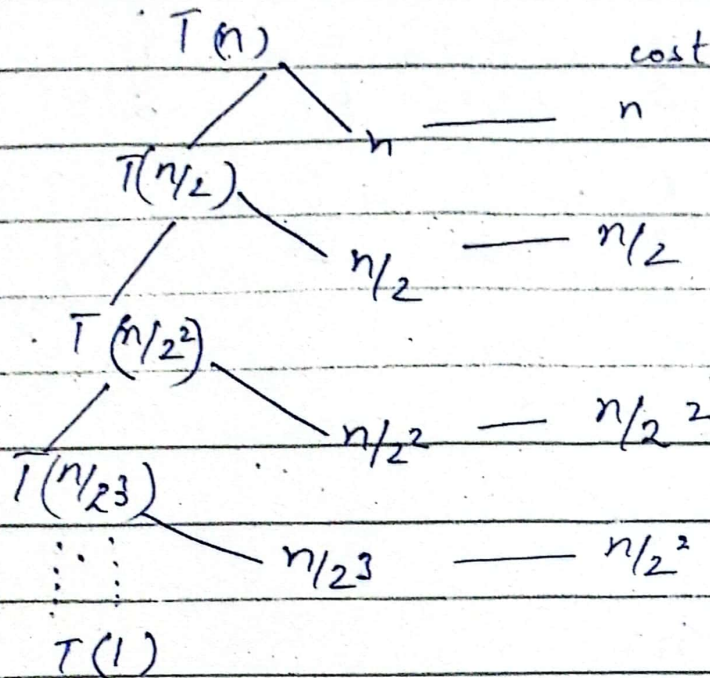
$$D^{-1} = \begin{bmatrix} 1 & -1 & 3 \\ 5 & 0 & 2 \\ 2 & 1 & 4 \end{bmatrix}$$

$$DD^{-1} = \begin{bmatrix} 1 & 5 & 2 \\ -1 & 0 & 1 \\ 3 & 2 & 4 \end{bmatrix} \begin{bmatrix} 1 & -1 & 3 \\ 5 & 0 & 2 \\ 2 & 1 & 4 \end{bmatrix}$$

$$= \begin{bmatrix} 1+25+4 & -1+0+2 & 3+10+8 \\ -1+0+2 & 1+0+1 & -3+0+4 \\ 3+10+8 & -3+0+4 & 9+4+16 \end{bmatrix}$$

$$= \begin{bmatrix} 30 & 1 & 21 \\ 1 & 2 & 1 \\ 21 & 1 & 29 \end{bmatrix} = 50 + 2 + 29 = 81$$

Solve $T(n) = T(n/2) + n$ using recursion tree method.



- value of node at i^{th} level = $n/2^i$
- value of node at last level = 1

$$\frac{n}{2^i} = 1$$

$$n = 2^i$$

$$\log n = i \log 2$$

$$i = \frac{\log n}{\log 2}$$

$$i = \log_2 n$$

cost of node at last level

$$= 1$$

no of nodes at i^{th} level
 $= 1$

$$T(n) = n + \frac{n}{2} + \frac{n}{2^2} + \frac{n}{2^3} + \dots + \frac{n}{2^{l-1}}$$

$$= n \left[1 + \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \dots + \frac{1}{2^{l-1}} \right]$$

$$= n \sum_{i=0}^{l-1} \frac{1}{2^i}$$

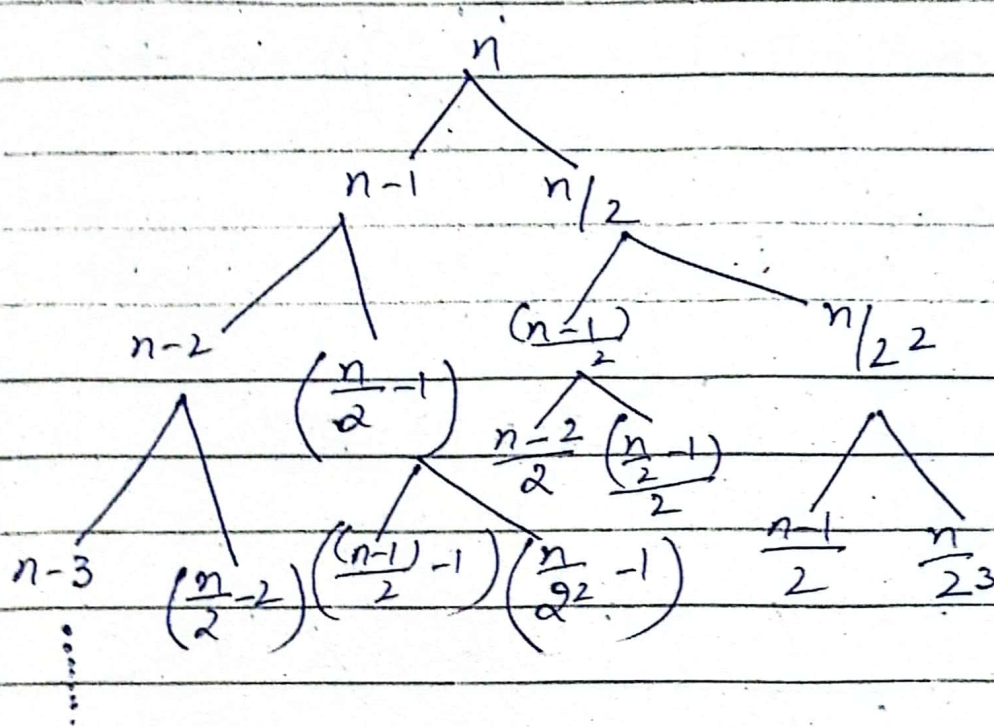
$$= n \times 1$$

$$= n$$

$$\in O(n)$$

Solve using recurrence relation
using tree method.

$$T(n) = T(n-1) + T(n/2) + n$$



$$T(1) \dots \dots \dots T(1)$$

Largest value of node at i^{th} level
= $n - i$

value of node at last level
= 1

$$n - i = 1$$

$$i = n - 1$$

no of nodes at i^{th} level = 2^i

no of nodes at last level = 2^{n-1}

$$\text{cost of node at last level} = 2^{n-1} \times T(1) = 2^{n-1}$$

$$\text{Total cost} = T(n) = \sum_{i=0}^{n-1-1} \left(\left(\frac{3}{2} \right)^i n - c \right) + 2^{n-1}$$

$$= n \sum_{i=0}^{n-1-1} \left(\frac{3}{2} \right)^i - c + 2^{n-1}$$

$$= n \left[\frac{1 - \left(\frac{3}{2} \right)^{n-1}}{\left(1 - \frac{3}{2} \right)} \right] - c + 2^{n-1}$$

$$= n \left[\frac{1 - \left(\frac{3}{2} \right)^{n-1}}{\left(-\frac{1}{2} \right)} \right] - c + 2^{n-1}$$

$$= n \left[2 \left(\left(\frac{3}{2} \right)^{n-1} - 1 \right) \right] - c + 2^{n-1}$$

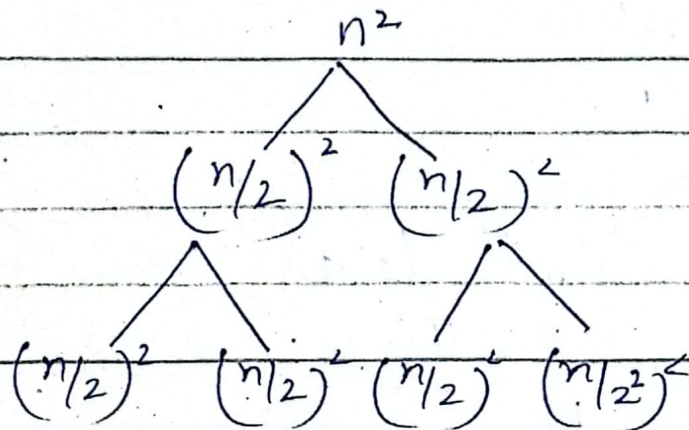
$$= n \left[\left(2^{-1} \right)^{n-1} \left(3^{n-1} \right) - 1 \right] - c + 2^{n-1}$$

$$= n \left(\frac{3}{2} \right)^{n-1} - n - c + 2^{n-1}$$

$$= \in O(2^n)$$

Solve using recursion tree method

$$T(n) = 2T(n/2) + n^2$$



$T(1) \cdot \cdot \cdot \cdot \cdot T(1)$

value of node at i^{th} level = $\left(\frac{n}{2^i}\right)^2$

value of node at last level = 1

$$\left(\frac{n}{2^i}\right)^2 = 1$$

$$\left(\left(\frac{n}{2^i}\right)^2\right)^{1/2} = (1)^{1/2}$$

$$\frac{n}{2^i} = 1$$

$$n = 2^i$$

$$\log n = i \log 2$$

$$i = \log_2 n$$

no of nodes at i^{th} level = 2^i

no of nodes at last level = $2^{\log_2 n}$

$$= n^{\log_2 2}$$

$$= n^1$$

$$= n$$

cost at last level

$$= n \times T(1)$$

$$= n$$

cost at level 0 = n^2

$$\text{at } L-1 = \frac{n^2}{4} + \frac{n^2}{4} = \frac{n^2}{2}$$

$$\text{at } L-3 = \frac{n^2}{4}$$

$$T(n) = \sum_{i=0}^{\log_2 n - 1} \left(\frac{n^2}{2^i} \right) + n$$

$$= n^2 \sum_{i=0}^{\log_2 n - 1} \left(\frac{1}{2} \right)^i + n$$

$$= n^2 \sum_{i=0}^{\log_2 n - 1} \left(\frac{1}{2} \right)^i + n$$

$$= n^2 \left[\frac{1}{1 - 1/2} \right] + n$$

$$= n^2 \left[\frac{1}{1/2} \right] + n = 2n^2 + n \in O(n^2)$$

Topic#:

Master theorem: (Shortcut method)

- Also called Recursion theorem.

Master theorem is a popular method for solving the recurrence relations.

Master theorem can be applied on specific problem not for all.

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the non-negative integers by the recurrence

$$T(n) = aT(n/b) + f(n)$$

n is the size of the problem.

a is the number of subproblems in the recursion.

n/b is the size of each subproblem.

$f(n)$ is the sum of dividing the problems and combining the solutions to the subproblems.

Cases:

i) $f(n) \in n^{\log_b a}$
 $T(n) = \Theta(n^{\log_b a})$

ii) $f(n) \in n^{\log_b a}$
 $T(n) = \Theta(n^{\log_b a} \cdot \log n)$

iii) $f(n) > n^{\log_b a}$
 $T(n) = \Theta(f(n))$

Solution is:

o $T(n) = n^{\log_b a} [U(n)]$

o $U(n)$ depends on $h(n)$

o $h(n) = \frac{f(n)}{n^{\log_b a}}$

o Relation between $f(n)$ & $h(n)$

if	$h(n)$	$U(n)$
	$n^\delta, \delta > 0$	$O(n^\delta)$
	$n^\delta, \delta < 0$	$O(1)$
	$(\log n)^i, i \geq 0$	$(\log n)^{i+1}$
		if $i < 0$

Definition of master theorem:

$\Rightarrow$ Let $a \geq 1$ and $b \geq 1$ are constants and $f(n)$ is a asymptotically +ve function

$\Rightarrow$ Let $T(n) = a T(n/b) + f(n)$ then master theorem provides the asymptotic bounds on $T(n)$ by these cases:

Case I:

- if $f(n) = O(n^{\log_b a - \epsilon})$
for some constant $\epsilon > 0$
then

$$T(n) = \Theta(n^{\log_b a})$$

Case II:

- if $f(n) = \Theta(n^{\log_b a})$
then

$$T(n) = \Theta(n^{\log_b a} \log n)$$

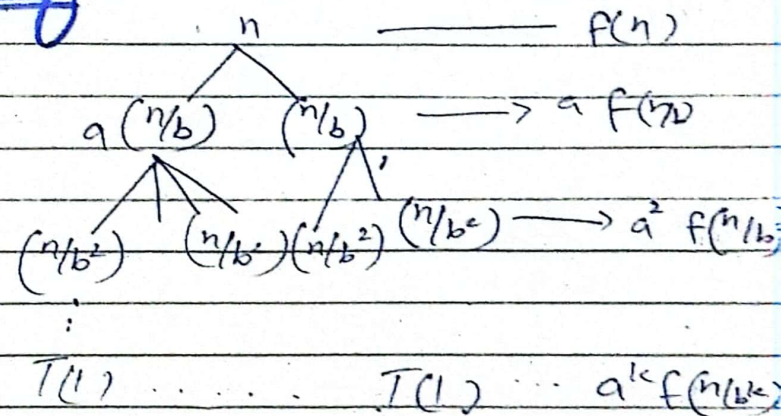
Case III:

- if $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$
and

$a f(n/b) \leq c f(n)$ for some constant $c < 1$ then

$$T(n) = \Theta(f(n))$$

Proof:



value of node at k^{th} level = $\frac{n}{b^k}$

value of node at last level = 1

$$\frac{n}{b^k} = 1$$

$$n = b^k$$

$$\log n = k \log b$$

$$k = \frac{\log n}{\log b}$$

$$k = \log_b n$$

cost at last level = $T(1) \times a \log_b n$

Total cost = $\sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right) + a \log_b n$ — (1)

$$T(n) = \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right) + a \log_b n$$

$$= a \log_b n$$

$$\log_b n = \frac{\log_a n}{\log_a b}$$

$$\therefore \log_k n = \frac{\log t^n}{\log t^k}$$

$$\therefore \log_k 5 = \frac{1}{\log_5 k}$$

$$\log_b n = \log_b a \cdot \log_a n$$

$$= \log_a n \log_b a$$

$$\log_b^n = \log_b^a \cdot \log_a^n$$

$$\log_b^n = \log_a^n \log_b^a$$

$$\therefore k \log n = \log n^k$$

$$\log_a^a \log_b^n = \log_a^n \log_b^a$$

$$\therefore \log_b^n \log_a^a = \log_a^a \log_b^n$$

$$a^{\log_b^n} = n^{\log_b^a}$$

$$a^{\log_b^n} = n^{\log_b^a}$$

$$T(n) = \sum_{i=0}^{\log_b^n - 1} a^i f\left(\frac{n}{b^i}\right) + n^{\log_b^a}$$

Case I: $f(n) \leq n^{\log_b^a - \epsilon}$ $f(n) = (n^{\log_b^a - \epsilon})$

$$= \sum_{i=0}^{\log_b^n - 1} a^i f\left(\frac{n}{b^i}\right) + n^{\log_b^a}$$

$$= \sum_{i=0}^{\log_b^n - 1} a^i \left(\frac{n}{b^i}\right)^{\log_b^a - \epsilon} + n^{\log_b^a}$$

$$= \sum_{i=0}^{\log_b^n - 1} a^i \sum_{i=0}^{\log_b^n - 1} a^i \cdot \frac{1}{b^{i(\log_b^a - \epsilon)}} + n^{\log_b^a}$$

$$= n^{\log_b^a - \epsilon} \sum_{i=0}^{\log_b^n - 1} a^i \frac{1}{b^{\log_b^a - \epsilon \cdot i}} + n^{\log_b^a}$$

$$= n^{\log_b^a - \epsilon} \sum_{i=0}^{\log_b^n - 1} a^i \frac{1}{b^{\log_b^a - \epsilon \cdot i}} + n^{\log_b^a}$$

$$= \frac{n \log_b a}{n^t} \sum_{i=0}^{\log_b n - 1} b^{it} + n \log_b a$$

$$= n \log_b a \left[\frac{1}{n^t} \frac{b^0 (b^{t \log_b n} - 1)}{b^t - 1} \right]$$

$$= n \log_b a \left[\frac{1}{n^t} \frac{b^0 [b^{\log_b n^t} - 1]}{b^t - 1} \right]$$

$$= n \log_b a \frac{1}{n^t} \frac{[n^t - 1]}{b^t - 1}$$

$$= \Theta(n \log_b a)$$



Example:

$$a) T(n) = 9T(n/3) + n$$

$$a = 9, b = 3, f(n) = n$$
$$= n \log_b a = n \log_3 9 = n^2$$

case I:

if

$$f(n) = O(n^{\log_b a - \epsilon})$$
$$= O(n^{\log_3 9 - \epsilon})$$
$$= O(n^{2 - \epsilon})$$
$$= O(n^{2-1}) \quad \because \epsilon = 1$$
$$= O(n)$$

then

$$T(n) \in \Theta(n^{\log_b a})$$
$$\in \Theta(n^2)$$

$$b) T(n) = T\left(\frac{2n}{3}\right) + 1$$

$$a = 1, b = 3/2, f(n) = 1$$

$$= n \log_b a$$

$$= n \log_{3/2} 1 = n^0 = 1$$

case II:

if

$$f(n) = \Theta(n^{\log_b a})$$

$$= \Theta(n^0) = \Theta(1)$$

then

$$\begin{aligned}T(n) &\in \Theta(n^{\log_b a} \cdot \log n) \\&= \Theta(1 \cdot \log n) \\&= \Theta(\log_2 n)\end{aligned}$$

$$T(n) = 2T(n/2) + n$$

$$\begin{aligned}a=2, \quad b=2, \quad f(n)=n \\= n^{\log_b a} = n^{\log_2 2} = n^1 = n\end{aligned}$$

Case II:

if

$$\begin{aligned}f(n) &= \Theta(n^{\log_b a} \log n) \\ \text{then } &= \Theta(n^{\log_b a} \log n) = \Theta(n \log n) \\ &\in O(n \log n)\end{aligned}$$

$$T(n) = 3T(n/4) + n \log n$$

$$\begin{aligned}a=3, \quad b=4, \quad f(n)=n \log n \\= n^{\log_b a} = n^{\log_4 3} = n^{0.75}\end{aligned}$$

Case III:

if

$$\begin{aligned}f(n) &= \Omega(n^{\log_b a + \epsilon}) \\&= \Omega(n^{0.75 + \epsilon}) \\&= \Omega(n^{0.75 + 0.25}) \quad \therefore \epsilon = 0.25 \\&= \Omega(n)\end{aligned}$$

And

$$\begin{aligned} a \sqrt[n]{(n/b)} &\leq c \sqrt[n]{n} \\ 3 \sqrt[n]{(n/4)} &\leq c n \log n \\ \frac{3n}{4} \log n/4 &\leq c n \log n \end{aligned}$$

$$\frac{3}{4} n \log n/4 \leq \frac{3}{4} n \log n \therefore c = 3/4$$

Then

$$\begin{aligned} T(n) &\in \Theta(f(n)) \\ &\in \Theta(n \log n) \end{aligned}$$

$$T(n) = 8T(n/2) + \Theta(n^2)$$

$$\begin{aligned} a=8, b=2, f(n) &= n^2 \\ &= n \log_b a = n \log_2 8 = n^3 \end{aligned}$$

Case I:

$$\begin{aligned} \text{if } f(n) &= O(n^{\log_b a - \epsilon}) \\ &= O(n^{\log_2 8 - \epsilon}) \\ &= O(n^{3-\epsilon}) \\ &= O(n^{3-1}) \\ &= O(n^2) \therefore \epsilon = 1 \end{aligned}$$

Then

$$\begin{aligned} T(n) &\in n^{\log_b a} \\ &\in \Theta(n^3) \end{aligned}$$

$$T(n) = 7T(n/2) + \Theta(n^2)$$

$$\begin{aligned} a &= 7, b = 2, f(n) = n^2 \\ &= n^{\log_b a} \\ &= n^{\log_2 7} \\ &= n^{2.8} \end{aligned}$$

Case I:

if

$$\begin{aligned} f(n) &= O(n^{\log_b a - \epsilon}) \\ &= O(n^{2.8 - \epsilon}) \\ &= O(n^{2.8 - 0.8}) \\ &= O(n^2) \end{aligned}$$

then

$$\begin{aligned} T(n) &\in \Theta(n^{\log_b a}) \\ &\in \Theta(n^{\log_2 7}) \\ &\in \Theta(n^{\log_2 7}) \\ &\in \Theta(n^{\log_2 7}) \end{aligned}$$

$$T(n) = 2T(n/4) + 1$$

$$\begin{aligned} a &= 2, b = 4, f(n) = 1 \\ &= n^{\log_b a} = n^{\log_4 2} = n^{\log_4 2} = \sqrt{n} \end{aligned}$$

Case I:

if

$$\begin{aligned} f(n) &= O(n^{\log_b a - \epsilon}) \\ &= O(n^{\log_4 2 - \epsilon}) \\ &= O(n^{\frac{1}{2} - \epsilon}) \end{aligned}$$

$$= O(n^{\frac{1}{2} - \frac{1}{2}})$$

$$= O(n^0)$$

$$= O(1)$$

then

$$T(n) \in \Theta(n^{\log_b a})$$

$$\in \Theta(n^{\log_4 2})$$

$$\in \Theta(\sqrt{n})$$

$$T(n) = 2T(n/4) + \sqrt{n}$$

$$a=2, b=4, f(n)=\sqrt{n}$$

$$= n^{\log_b a} = n^{\log_4 2} = n^{1/2} = \sqrt{n}$$

case II:

i.e.

$$f(n) = \Theta(n^{\log_b a})$$

$$= \Theta(n^{\log_4 2})$$

$$= \Theta(n^{1/2})$$

then

$$T(n) \in \Theta(n^{\log_b a} \cdot \log n)$$

$$\in \Theta(\sqrt{n} \cdot \log n)$$

$$T(n) = 2T(n/4) + n$$

$$\begin{aligned} a &= 2, b = 4, f(n) = n \\ &= n \log_b a \\ &= n \log_4 2 \\ &= n^{1/2} = \sqrt{n} \end{aligned}$$

Case III:

$$\begin{aligned} \text{if } f(n) &= \Omega(n^{\log_b a + \epsilon}) \\ &= \Omega(n^{1/2 + \epsilon}) \\ &= \Omega(n^{1/2 + 1/2}) \quad \therefore \epsilon = 1/2 \\ &= \Omega(n) \end{aligned}$$

And

$$\begin{aligned} a f(n/b) &\leq c f(n) \\ 2 f(n/4) &\leq c n \\ 2 \left(\frac{n}{4} \right) &\leq c n \end{aligned}$$

$$n/2 \leq c n$$

$$\frac{1}{2} n \leq c n \quad \therefore c = 1/2$$

$$\frac{1}{2} n \leq \frac{1}{2} n$$

Hence

$$\begin{aligned} T(n) &\in \Theta(f(n)) \\ &\in \Theta(n) \end{aligned}$$

$$T(n) = 2T(n/4) + n^2$$

$$a=2, b=4, f(n)=n^2$$

$$= n^{\log_b a}$$

$$= n^{\log_4 2} = \sqrt{n}$$

Case III:

if

$$f(n) = \Omega(n^{\log_b a + \epsilon})$$

$$= \Omega(n^{\log_4 2 + \epsilon})$$

$$= \Omega(n^{1/2 + \epsilon})$$

$$= \Omega(n^{1/2 + 1/2}) \quad \therefore \epsilon = 1/2$$

$$= \Omega(n^1)$$

$$= \Omega(n)$$

And

$$a f(n/b) \leq c f(n)$$

$$2 f(n/4) \leq c n^2$$

$$2 \left(\frac{n}{4} \right)^2 \leq c n^2$$

$$2 \left(\frac{n^2}{16} \right) \leq c n^2$$

$$\frac{1}{8} n^2 \leq c n^2$$

$$\therefore c \geq 1/8$$

$$\frac{1}{8} n^2 \leq \frac{1}{8} n^2$$

then

$$T(n) = \Theta(f(n))$$

$$= \Theta(n^2)$$

$$T(n) = T(n/2) + 1$$

$$a=1, b=2, f(n)=1$$

$$= n^{\log_b a}$$

$$= n^{\log_2 1}$$

$$= n^0$$

$$= 1$$

case II:

i/j

$$\begin{aligned} f(n) &= \Theta(n^{\log_b a}) \\ &= \Theta(n^{\log_2 1}) \\ &= \Theta(n^0) \\ &= \Theta(1) \end{aligned}$$

then

$$\begin{aligned} T(n) &\in \Theta(n^{\log_b a} \cdot \log n) \\ &\quad \Theta(n^0 \cdot \log n) \\ &\quad \Theta(1 \cdot \log n) \\ &\quad \Theta(\log n) \end{aligned}$$

Q#: Can the master method be applied to the recurrence $T(n) = 4T(n/2) + n^2 \log n$? Why or why not? Give an asymptotic upper bound for this recurrence.

$$\begin{aligned}
 a &= 4, b = 2, f(n) = n^2 \lg n \\
 &= n \lg b^a \\
 &= n \lg 2^4 \\
 &= n^2
 \end{aligned}$$

Case 1:

$$\begin{aligned}
 \text{if } f(n) &= \Omega(n^{\log_b a - \epsilon}) \\
 &= \Omega(n^{\log_2 4 - \epsilon}) \\
 &= \Omega(n^{2 - \epsilon}) \\
 &= \Omega(n^{2-1}) \\
 &= \Omega(n)
 \end{aligned}$$

And

$f(n) \neq n^2 \lg n \neq O(n^{2-\epsilon}) \neq \Omega(n^{2-\epsilon})$, so we cannot apply master theorem

$$\begin{aligned}
 T(n) &\leq 4T(n/2) + n^2 \lg n \\
 &\leq 4c(n/2)^2 \lg^2(n/2) + n^2 \lg n \\
 &\leq cn^2 \lg(n/2) \lg n - cn^2 \lg(n/2) \lg 2 + n^2 \lg n \\
 &\leq cn^2 \lg^2 n - cn^2 \lg n \lg 2 - cn^2 \lg(n/2) + n^2 \lg n \\
 &\leq cn^2 \lg^2 n + (1-c)n^2 \lg n - cn^2 \lg(n/2) \\
 &\leq cn^2 \lg^2 n - cn^2 \lg(n/2) \\
 &\leq cn^2 \lg^2 n
 \end{aligned}$$

$$T(n) = 2^n T(n/2) + 2^n$$

Master method cannot be apply because a is not constant.

$$T(n) = 2T(n/2) + n/\log n$$

Master method cannot be apply (non-polynomial difference between $f(n)$ and $n^{\log_b a}$)

$$T(n) = 0.5T(n/2) + 1/n$$

can not apply because ($a < 1$). it should be greater than 1.

$$T(n) = 64T(n/8) - n^2 \log n$$

Can not apply because $f(n)$ is not positive.

$$T(n) = T(n/2) + n(2 - \cos n)$$

Does not apply, we are in case 3, but the regularity condition is violated (consider $n = 2\pi k$, where k is odd and arbitrarily large. For any such choice of n , you can show that $c \geq 3/2$ thereby violating the regularity condition).

$$T(n) = T(n/2) + \log n \quad \text{if } n \geq 2$$

we cannot apply master's theorem
but this is special case
 $n = 2^m$

$$T(2^m) = T(2^{m/2}) + \log 2^m$$

$$= T(2^{m/2}) + m \log 2$$

$$T(2^n) = T(2^{n/2}) + m$$

$$T(2^m) = S(m)$$

$$S(m) = S(m/2) + m$$

$$a = 1, b = 2, f(n) = m$$

$$= n^{\log_b a}$$

$$= n^{\log_2 1}$$

$$= n^0 = 1$$

$$\log n = m \log 2$$

$$m = \frac{\log n}{\log 2} = \log_2 n$$

case III:

if

$$f(n) = \Omega(n^{\log_b a + \epsilon})$$

$$= \Omega(n^{\log_2 1 + \epsilon})$$

$$= \Omega(n^{0 + \epsilon})$$

$$= \Omega(n^\epsilon)$$

Ans

$$a f(n/b) \leq c f(n)$$

$$1 f(n/2) \leq c f(n)$$

$$1 (\log_2 n/2) \leq c \log_2 n$$

$$\frac{1}{2} \log_2 n \leq c \log_2 n$$

$$\frac{1}{2} \log_2^n \leq \frac{1}{2} \log_2^n \therefore c = 1/2$$

then

$$T(n) \in \Theta(f(n))$$

$$\in \Theta(\log^2 n)$$

$$T(n) = 2T(\sqrt{n}) + \lg n$$

$$m = \lg n$$

$$T(2^m) = 2T(2^{m/2}) + m$$

$$S(m) = T(2^m)$$

$$S(m) = 2S(m/2) + m$$

$$S(m) = O(m \lg m)$$

changing back from $S(m)$ to $T(m)$

$$T(n) = T(2^m) = S(m) = O(m \lg m)$$

$$= O(\lg n \lg \lg n)$$